

UNIT 4

Exception Handling

An Exception is an unwanted error or hurdle that a program throws while compiling. There are various methods to handle an exception which is termed exceptional handling.

Exception Handling and how we catch base and derived classes as an exception in C++:

- If both base and derived classes are caught as exceptions, then the catch block of the derived class must appear before the base class.
- If we put the base class first then the derived class catch block will never be reached.

// C++ Program to demonstrate a catching Base Exception

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
};
```

```
class Derived : public Base {
```

```
};
```

```
int main()
```

```
{
```

```
    Derived d;
```

```
    try {
```

```
        throw d;
```

```
    }
```

```

    catch (Base b) {

        cout << "Caught Base Exception";

    }

    catch (Derived d) {

        // This 'catch' block is NEVER executed

        cout << "Caught Derived Exception";

    }

    getchar();

    return 0;

}

```

Output

Caught Base Exception

*The **output** of the above C++ code:*

prog.cpp: In function 'int main()':

prog.cpp:20:5: warning: exception of type 'Derived' will be caught

```

    catch (Derived d) {

```

```

    ^

```

prog.cpp:17:5: warning: by earlier handler for 'Base'

```

    catch (Base b) {

```

In the above C++ code, if we change the order of catch statements then both catch statements become reachable.

The modified program:

```
// C++ Program to demonstrate a catching of derived exception and printing it successfully
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {};
```

```
class Derived : public Base {};
```

```
int main()
```

```
{
```

```
    Derived d;
```

```
    // Some other functionalities
```

```
    try {
```

```
        // Monitored code
```

```
        throw d;
```

```
    }
```

```
    catch (Derived d) {
```

```
        cout << "Caught Derived Exception";
```

```
    }
```

```
    catch (Base b) {
```

```
        cout << "Caught Base Exception";
```

```
    }
```

```
    getchar(); // To read the next character
```

```
    return 0;
```

```
}
```

Output

Caught Derived Exception

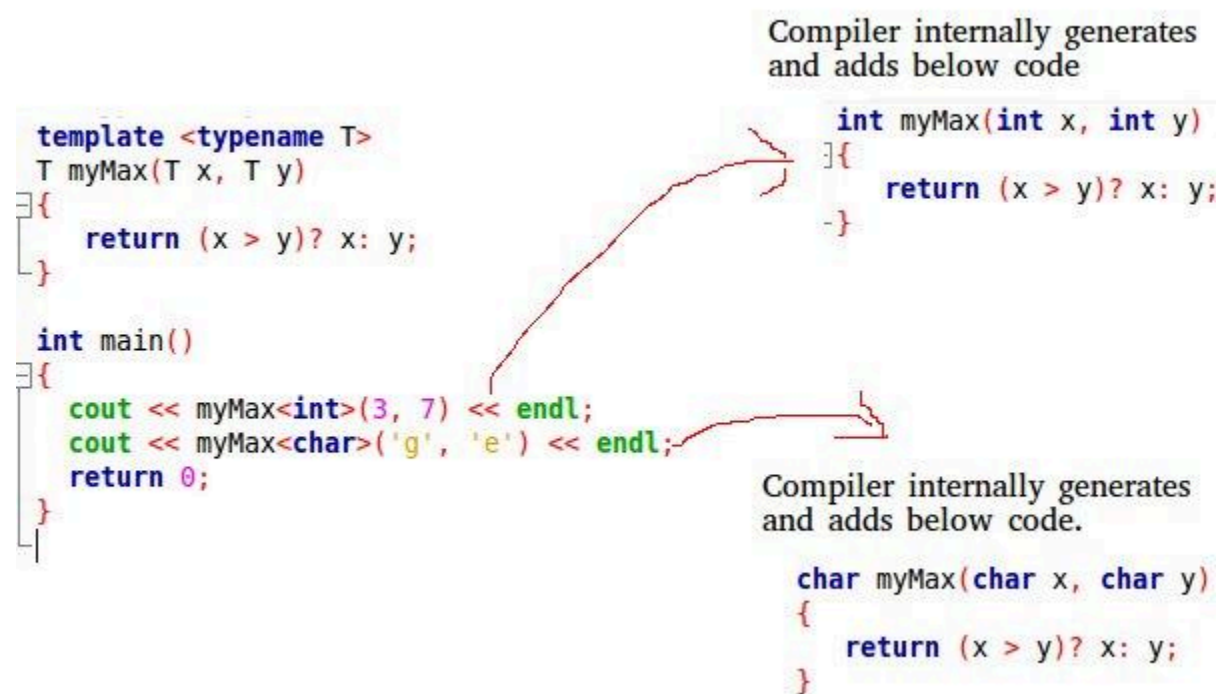
Templates in C++

A **template** is a simple yet very powerful tool in C++. The simple idea is to pass the data type as a parameter so that we don't need to write the same code for different data types. For example, a software company may need to sort() for different data types. Rather than writing and maintaining multiple codes, we can write one sort() and pass the datatype as a parameter.

C++ adds two new keywords to support templates: '*template*' and '*type name*'. The second keyword can always be replaced by the keyword '**class**'.

How Do Templates Work?

Templates are expanded at compile time. This is like macros. The difference is, that the compiler does type-checking before template expansion. The idea is simple, source code contains only function/class, but compiled code may contain multiple copies of the same function/class.



Function Templates

We write a generic function that can be used for different data types. Examples of function templates are sort(), max(), min(), printArray().

```
// C++ Program to demonstrate
// Use of template
#include <iostream>
using namespace std;

// One function works for all data types. This would work
// even for user defined types if operator '>' is overloaded
template <typename T> T myMax(T x, T y)
{
    return (x > y) ? x : y;
}

int main()
{
    // Call myMax for int
    cout << myMax<int>(3, 7) << endl;
    // call myMax for double
    cout << myMax<double>(3.0, 7.0) << endl;
    // call myMax for char
    cout << myMax<char>('g', 'e') << endl;

    return 0;
}
```

Output

7

7

g

Class Templates

Class templates like function templates, class templates are useful when a class defines something that is independent of the data type. Can be useful for classes like LinkedList, BinaryTree, Stack, Queue, Array, etc.

```
// C++ Program to implement
```

```
// template Array class
```

```
#include <iostream>
```

```
using namespace std;
```

```
template <typename T> class Array {
```

```
private:
```

```
    T* ptr;
```

```
    int size;
```

```
public:
```

```
    Array(T arr[], int s);
```

```
    void print();
```

```
};
```

```
template <typename T> Array<T>::Array(T arr[], int s)
```

```
{
```

```
    ptr = new T[s];
```

```
    size = s;
```

```
        for (int i = 0; i < size; i++)
            ptr[i] = arr[i];
    }

template <typename T> void Array<T>::print()
{
    for (int i = 0; i < size; i++)
        cout << " " << *(ptr + i);
    cout << endl;
}

int main()
{
    int arr[5] = { 1, 2, 3, 4, 5 };
    Array<int> a(arr, 5);
    a.print();
    return 0;
}
```

Standard Template Library

In C++, the Standard Template Library (STL) provides a set of programming tools to implement algorithms and data structures like vectors, lists, queues, etc.

STL implements these data structures and algorithms using general-purpose classes and functions that have been tested rigorously.

C++ STL has 3 major components:

- Containers
- Iterators
- Algorithms

In addition to these, STL also provides several other features, including function objects, smart pointers, and exception handling mechanisms.

It is a generalized library and so, its components are parameterized.

The C++ Standard Template Library (STL) is a collection of algorithms, data structures, and other components that can be used to simplify the development of C++ programs. The STL provides a range of containers, such as vectors, lists, and maps, as well as algorithms for searching, sorting and manipulating data.

One of the key benefits of the STL is that it provides a way to write generic, reusable code that can be applied to different data types. This means that you can write an algorithm once, and then use it with different types of data without having to write separate code for each type.

The STL also provides a way to write efficient code. Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.

Some of the key components of the STL include:

1. Containers: The STL provides a range of containers, such as vector, list, map, set, and stack, which can be used to store and manipulate data.
2. Algorithms: The STL provides a range of algorithms, such as sort, find, and binary_search, which can be used to manipulate data stored in containers.
3. Iterators: Iterators are objects that provide a way to traverse the elements of a container. The STL provides a range of iterators, such as forward_iterator, bidirectional_iterator, and random_access_iterator, that can be used with different types of containers.
4. Function Objects: Function objects, also known as functors, are objects that can be used as function arguments to algorithms. They provide a way to pass a function to an algorithm, allowing you to customize its behavior.
5. Adapters: Adapters are components that modify the behavior of other components in the STL. For example, the reverse_iterator adapter can be used to reverse the order of elements in a container.

By using the STL, you can simplify your code, reduce the likelihood of errors, and improve the performance of your programs.

STL has 4 components:

- Algorithms
- Containers
- Functors
- Iterators

1. Algorithms

The header algorithm defines a collection of functions specially designed to be used on a range of elements. They act on containers and provide means for various operations for the contents of the containers.

2. Containers

Containers or container classes store objects and data. There are in total seven standard “first-class” container classes and three container adaptor classes and only seven header files that provide access to these containers or container adaptors.

- Sequence Containers: implement data structures that can be accessed in a sequential manner.
 - **vector**
 - list
 - deque
 - arrays
 - forward_list(Introduced in C++11)
- Container Adaptors: provide a different interface for sequential containers.
 - queue
 - priority_queue
 - stack
- Associative Containers: implement sorted data structures that can be quickly searched ($O(\log n)$ complexity).
 - set
- Unordered Associative Containers: implement unordered data structures that can be quickly searched
 - unordered_set (Introduced in C++11) etc.

3. Functors

The STL includes classes that overload the function call operator. Instances of such classes are called function objects or functors. Functors allow the working of the associated function to be customized with the help of parameters to be passed.

4. Iterators

As the name suggests, iterators are used for working on a sequence of values. They are the major feature that allows generality in STL.

Advantages of the C++ Standard Template Library (STL):

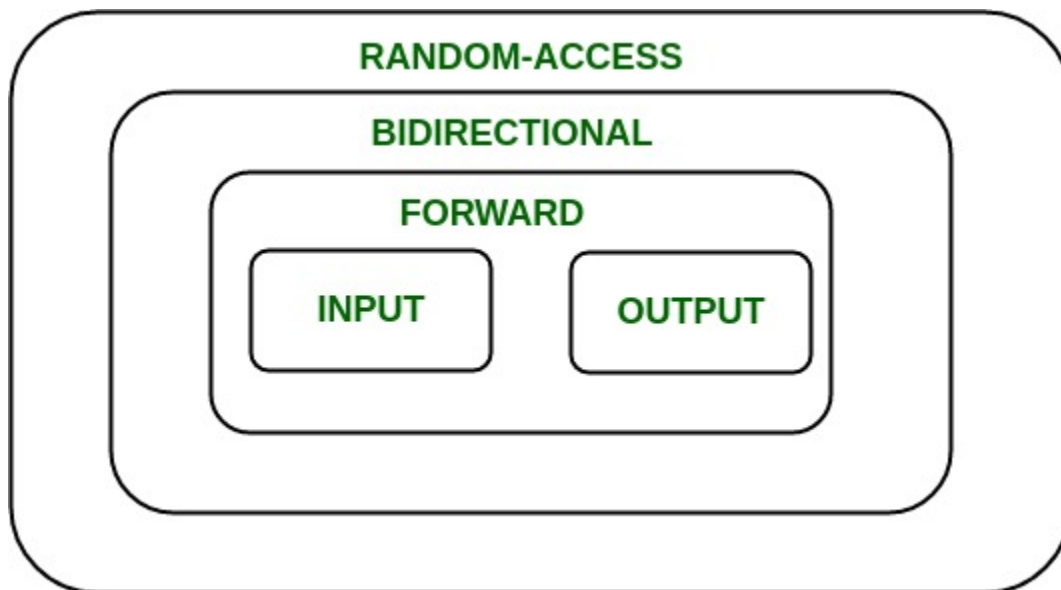
1. **Reusability:** One of the key advantages of the STL is that it provides a way to write generic, reusable code that can be applied to different data types. This can lead to more efficient and maintainable code.
2. **Efficient algorithms:** Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.
3. **Improved code readability:** The STL provides a consistent and well-documented way of working with data, which can make your code easier to understand and maintain.
4. **Large community of users:** The STL is widely used, which means that there is a large community of developers who can provide support and resources, such as tutorials and forums.

Disadvantages of the C++ Standard Template Library (STL):

1. Learning curve: The STL can be difficult to learn, especially for beginners, due to its complex syntax and use of advanced features like iterators and function objects.
2. Lack of control: When using the STL, you have to rely on the implementation provided by the library, which can limit your control over certain aspects of your code.
3. Performance: In some cases, using the STL can result in slower execution times compared to custom code, especially when dealing with small amounts of data.

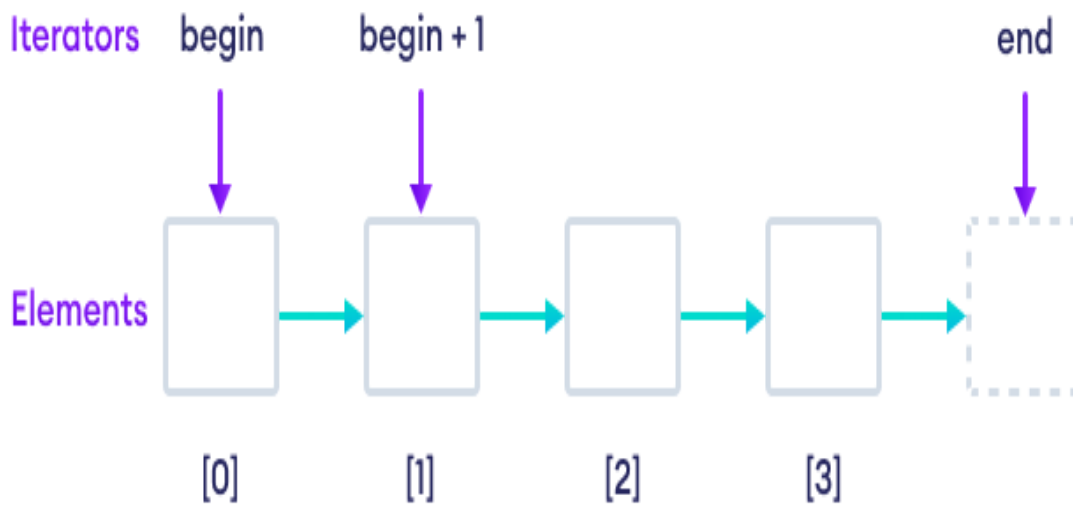
Iterators in C++

An **iterator** is an object (like a pointer) that points to an element inside the container. We can use iterators to move through the contents of the container. They can be visualized as something similar to a pointer pointing to some location and we can access the content at that particular location using them. Iterators play a critical role in connecting algorithms with containers along with the manipulation of data stored inside the containers. The most obvious form of an iterator is a pointer. A pointer can point to elements in an array and can iterate through them using the increment operator (++). But, all iterators do not have similar functionality as that of pointers. Depending upon the functionality of iterators they can be classified into five categories, as shown in the diagram below with the outer one being the most powerful one and consequently the inner one is the least powerful in terms of functionality.



Suppose we have a vector named `nums` of size 4. Then, `begin()` and `end()` are member functions that return iterators pointing to the beginning and end of the vector respectively.

- `nums.begin()` points to the first element in the vector i.e 0th index
- `nums.begin() + i` points to the element at the *i*th index.
- `nums.end()` points to one element past the final element in the vector



```
#include <iostream>

#include<vector>

using namespace std;


int main() {

    vector <string> languages = {"Python", "C++", "Java"};


    // create an iterator to a string vector

    vector<string>::iterator itr;


    // iterate over all elements

    for (itr = languages.begin(); itr != languages.end(); itr++) {

        cout << *itr << ", ";

    }


    return 0;

}
```

Output Python, C++, Java,

In the above example, we have used an iterator `itr` to iterate over a vector named `languages`.

Here,

- `itr = languages.begin()` - assigns the iterator pointing to the first vector element to `itr`
- `itr != languages.end()` - checks whether `itr` has reached the end of the vector
- `itr++` - increments `itr` to the next position
- `*itr` - returns the vector element at the `itr` position

Iterator Fundamental Operations

Some fundamental operations that can be performed on iterators are shown in the table below.

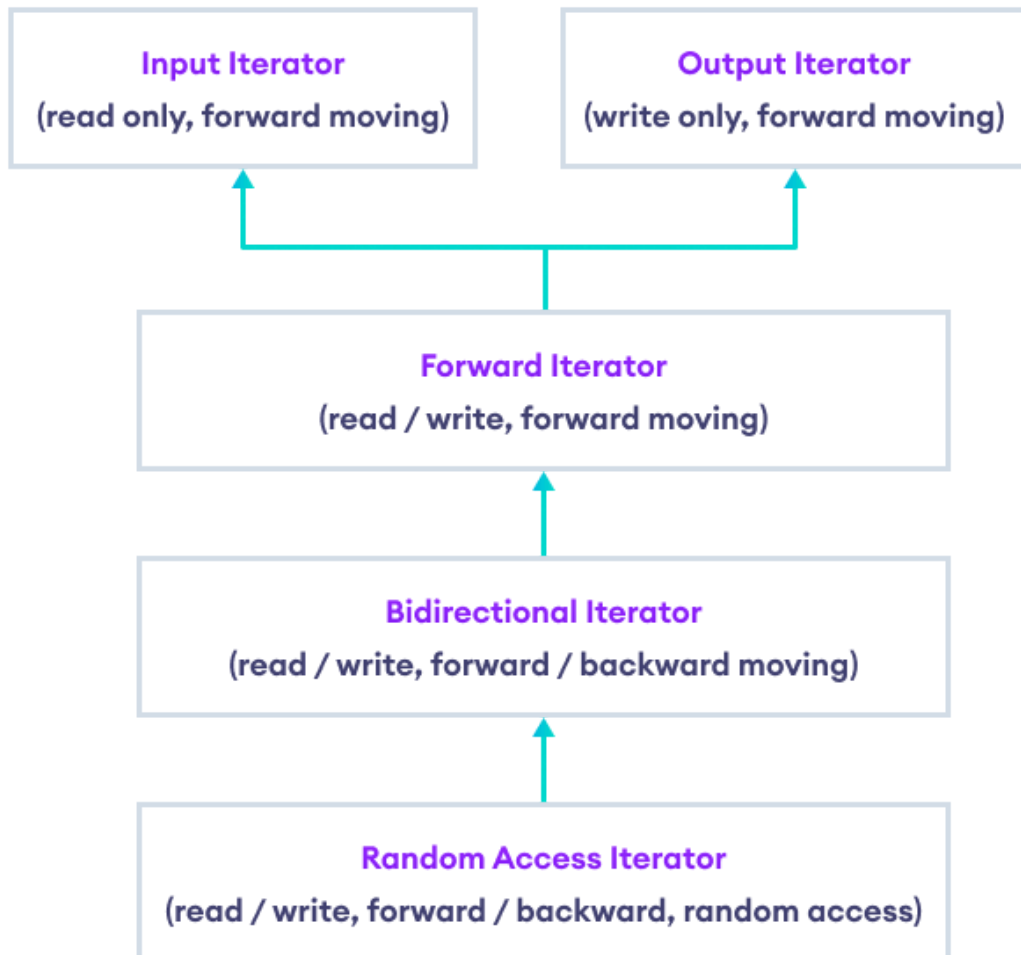
Operations	Description
<code>*itr</code>	returns the element at the current position
<code>itr->m</code>	returns the member value <code>m</code> of the object pointed by the iterator and is equivalent to <code>(*itr).m</code>
<code>++itr</code>	moves iterator to the next position
<code>--itr</code>	moves iterator to the previous position
<code>itr + i</code>	moves iterator by <code>i</code> positions
<code>itr1 == itr2</code>	returns true if the positions pointed by the iterators are the same
<code>itr1 != itr2</code>	returns true if the positions pointed by the iterators are not the same
<code>itr = itr1</code>	assigns the position pointed by <code>itr1</code> to the <code>itr</code> iterator

Now each one of these iterators are not supported by all the containers in STL, different containers support different iterators, like vectors support Random-access iterators, while lists support bidirectional iterators.

CONTAINER	TYPES OF ITERATOR SUPPORTED
Vector	Random-Access
List	Bidirectional
Deque	Random-Access
Map	Bidirectional
Multimap	Bidirectional
Set	Bidirectional
Multiset	Bidirectional
Stack	No iterator Supported
Queue	No iterator Supported
Priority-Queue	No iterator Supported

ITERATORS	PROPERTIES				
	ACCESS	READ	WRITE	ITERATE	COMPARE
Input	->	= *i		++	==, !=
Output			*i=	++	
Forward	->	= *i	*i=	++	==, !=
Bidirectional		= *i	*i=	++, --	==, !=, <, >
Random-Access	->, []	= *i	*i=	++, --, +=, -=, +, -	==, !=, <, >, <=, >=

Types of iterators: Based upon the functionality of the iterators, they can be classified into five major categories:



1. **Input Iterators:** They are the weakest of all the iterators and have very limited functionality. They can only be used in single-pass algorithms, i.e., those algorithms which process the container sequentially, such that no element is accessed more than once.

```
// create an input iterator to read values from cin
```

```
istream_iterator<int> input_itr(cin);
```

Here, we have created an input iterator named `input_itr` of type `int` that reads input values from the standard input stream `cin`.

Note: Input and output iterators are part of C++ STL iterators and are defined in the `<iterator>` header file.

2. **Output Iterators:** Just like input iterators, they are also very limited in their functionality and can only be used in single-pass algorithms, but not for accessing elements, but for being assigned elements.

```
// create an output iterator to write integers to the console
```

```
ostream_iterator<int> output_itr(cout, " ");
```

3. **Forward Iterator:** They are higher in the hierarchy than input and output iterators, and contain all the features present in these two iterators. But, as the name suggests, they also can only move in a forward direction and that too one step at a time.

```
#include <iostream>
```

```
#include <forward_list>
```

```
using namespace std;
```

```
int main() {
```

```
    forward_list<int> nums{1, 2, 3, 4};
```

```
    // initialize an iterator to point to beginning of a forward list
```

```
    forward_list<int>::iterator itr = nums.begin();
```

```
    while (itr != nums.end()) {
```

```
        // access iterator value using indirection operator
```

```
        int original_value = *itr;
```

```
        // assign new value using indirection operator
```

```
        *itr = original_value * 2
```

```

        // forward the iterator to next position

        itr++;

    }

    // display the contents of nums

    for (int num: nums) {

        cout << num << ", ";

    }

    return 0;

}

```

Output

```
2, 4, 6, 8,
```

4. **Bidirectional Iterators:** They have all the features of forward iterators along with the fact that they overcome the drawback of forward iterators, as they can move in both the directions, that is why their name is bidirectional. We can iterate forward using ++, backward using --, and read and write values using * or read and write member values using ->. The iterators of the container classes list, set, multiset, map, and multimap are bidirectional iterators.

```
#include <iostream>
```

```
#include <list>
```

```
using namespace std;
```

```
int main() {
```

```
    list<int> nums {1, 2, 3, 4, 5};
```

```
    // initialize iterator to point to beginning of nums
```

```
    list<int>::iterator itr = nums.begin();
```

```
    cout << "Moving forward: " << endl;
```

```
    // display the elements in forward order
```

```
    while (itr != nums.end()) {
```

```
        cout << *itr << ", ";
```

```
    // move iterator by 1 position forward
```

```
        itr++;
```

```
    }
```

```

cout << endl << "Moving backward: " << endl;

// display the elements in backward order

while (itr != nums.begin()) {

    if (itr != nums.end()) {

        cout << *itr << ", ";

    }

    // move iterator by 1 position backward

    itr--;

}

cout << *itr << endl;

return 0;

}

```

Output

Moving forward:

1, 2, 3, 4, 5,

Moving backward:

5, 4, 3, 2, 1

5. **Random-Access Iterators:** They are the most powerful iterators. They are not limited to moving sequentially, as their name suggests, they can randomly access any element inside the container. They are the ones whose functionality is the same as pointers. The iterators of the container classes vector, deque, array, and iterators of strings are random-access iterators.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    // create a vector
```

```
    vector<int> vec{1, 2, 3, 4};
```

```
    // create iterators to point to the first and the last elements
```

```
    vector<int>::iterator itr_first = vec.begin();
```

```
    vector<int>::iterator itr_last = vec.end() - 1;
```

```
    // display the vector elements
```

```
    cout << "First Element: " << *itr_first << endl;
```

```
cout << "Second Element: " << itr_first[1] << endl;
```

```
cout << "Second Last Element: " << *(itr_last - 1) << endl;
```

```
cout << "Last Element: " << *(itr_last) << endl;
```

```
return 0;
```

```
}
```

Output

```
First Element: 1
```

```
Second Element: 2
```

```
Second Last Element: 3
```

```
Last Element: 4
```

Benefits of Iterators

There are certainly quite a few ways which show that iterators are extremely useful to us and encourage us to use it profoundly. Some of the benefits of using iterators are as listed below:

- **Convenience in programming:** It is better to use iterators to iterate through the contents of containers as if we will not use an iterator and access elements using [] operator, then we need to be always worried about the size of the container, whereas with iterators we can simply use member function end() and iterate through the contents without having to keep anything in mind.

- **Code reusability:** Now consider if we make `v` a list in place of vector in the above program and if we were not using iterators to access the elements and only using `[ ]` operator, then in that case this way of accessing was of no use for list (as they don't support random-access iterators). However, if we were using iterators for vectors to access the elements, then just changing the vector to list in the declaration of the iterator would have served the purpose, without doing anything else. So, iterators support reusability of code, as they can be used to access elements of any container.
- **Dynamic processing of the container:** Iterators provide us the ability to dynamically add or remove elements from the container as and when we want with ease.
- **Support for algorithms:** The C++ Standard Library provides a wide range of algorithms that can be used with iterators, such as `std::find()`, `std::sort()`, and `std::accumulate()`. These algorithms allow us to perform common operations on containers.
- **Memory efficiency:** Iterators allow us to process large datasets one element at a time without loading the entire dataset at once, resulting in memory efficiency.
- **Consistency:** Iterators provide a way to access and manipulate data of different types of containers in a consistent way. We can use the same iterator syntax for vectors, lists, sets, or any other container.
- **Simplification of code:** Iterators can simplify the code by hiding the details of iterating over the container, thus making the code more readable.

// C++ program to demonstrate iterators

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// Declaring a vector
```

```
vector<int> v = { 1, 2, 3 };
```

```
// Declaring an iterator
```

```
vector<int>::iterator i;
```

```
int j;
```

```
cout << "Without iterators = ";
```

```
// Accessing the elements without using iterators
```

```
for (j = 0; j < 3; ++j)
```

```
{
```

```
    cout << v[j] << " ";
```

```
}
```

```
cout << "\nWith iterators = ";
```

```
// Accessing the elements using iterators
```

```
for (i = v.begin(); i != v.end(); ++i)
```

```
{
```

```
    cout << *i << " ";
```

```
}
```

```
// Adding one more element to vector
```

```
v.push_back(4);
```

```
cout << "\nWithout iterators = ";
```

```
// Accessing the elements without using iterators
```

```
for (j = 0; j < 4; ++j)
```

```
{
```

```
    cout << v[j] << " ";
```

```
}
```

```
cout << "\nWith iterators = ";
```

```

// Accessing the elements using iterators

for (i = v.begin(); i != v.end(); ++i)

{

    cout << *i << " ";

}

return 0;

}

```

Output:

Without iterators = 1 2 3

With iterators = 1 2 3

Without iterators = 1 2 3 4

With iterators = 1 2 3 4

Explanation: As can be seen in the above code that without using iterators we need to keep track of the total elements in the container. In the beginning there were only three elements, but after one more element was inserted into it, accordingly the for loop also had to be amended, but using iterators, both the time and the for loop remained the same. So, the iterator eased our task.

```

// C++ program to demonstrate iterators

```

```

#include <iostream>

```

```

#include <vector>

using namespace std;

int main()
{
    // Declaring a vector
    vector<int> v = { 1, 2, 3 };

    // Declaring an iterator
    vector<int>::iterator i;

    int j;

    // Inserting element using iterators
    for (i = v.begin(); i != v.end(); ++i) {
        if (i == v.begin()) {
            i = v.insert(i, 5);
            // inserting 5 at the beginning of v
        }
    }

    // v contains 5 1 2 3

    // Deleting a element using iterators
    for (i = v.begin(); i != v.end(); ++i) {
        if (i == v.begin() + 1) {
            i = v.erase(i);
            // i now points to the element after the

```

```

        // deleted element
    }
}

// v contains 5 2 3

// Accessing the elements using iterators
for (i = v.begin(); i != v.end(); ++i) {
    cout << *i << " ";
}

return 0;
}

```

Output:

5 2 3

Explanation: As seen in the above code, we can easily and dynamically add and remove elements from the container using an iterator, however, doing the same without using them would have been very tedious as it would require shifting the elements every time before insertion and after deletion.

Iterators	Pointers
Used to iterate over containers like vectors, lists, etc.	Used to point to a memory location.
Can be invalidated when the underlying container is modified.	Remains valid even if the underlying data structure is modified.
Can be used with algorithms like <code>std::find()</code> , <code>std::sort()</code> , etc.	Cannot be used with algorithms like <code>std::find()</code> , <code>std::sort()</code> , etc.
Can be used with ranged for loops.	Cannot be used with ranged for loops.
Can be categorized as input, output, forward, bidirectional, or random access iterators based on their capabilities.	Pointers are inherently random access.

Introduction to C++ hash

Hashing in C++ STL is a technique that maps a key with the associated hash value. The elements in an array can be referred to as the keys, which will have a hash value that helps in finding these keys in the hash table easily. This helps in faster access to the required element, and the efficiency might vary based on the selection of the hash function.

Why is Hashing so Important? Although the most popular data structures, such as arrays and linked lists, can still be utilized to search for the necessary key, they could be more efficient.

Arrays can be helpful when we need to randomly access any element from a particular index, but for this purpose, we need to know the index of the specific element.

This is where hashing comes into the picture. The hash function finds the hash value where a key should be stored. When we need to fetch this key, it can be easily done with the help of its hash value.

As an example, the hashing technique makes finding a person's contact information easier with the help of their names.

How to Work with Hash Function in C++?

The hash function yields an integer value from the given key, be it a character, string, integer, etc., and associates this index with the actual key.

The syntax to create a template for hash class:

```
template <class key> struct hash;
```

Now, to create objects with the hash class, we can use the following syntax:

```
hash<class template> object-name
```

The hash class also has a single member function, `operator()`, that returns the object's hash value.

```
// Initializing character.
```

```
char ch = 'n';
```

```
// Creating object of the hash class.
```

```
hash<char> ch_hash;
```

```
// Using operator() to find hash value.
```

```
cout << ch_hash(ch);
```

But How Does Hashing Work? There are two heuristic techniques (an approach that finds shortcuts to a solution that might not be optimal but can be reached within the required deadline) to create a hash function, hashing by division and hashing by multiplication.

A good hash function has the following properties:

- It should be efficient for computation.
- All the keys should be uniformly distributed in the hash table.

52	42	44	66	25	29	32
----	----	----	----	----	----	----

Array

SCALER
Topics

Suppose we have 7 elements in an array. We must create a hash table where these elements will be stored for efficient access. When we come across the first element, 52, we'll perform the following operation to get its index.

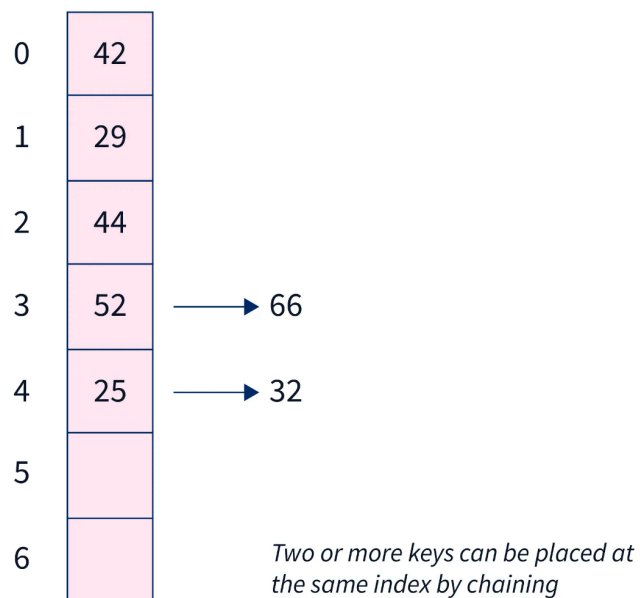
$\text{hash_index}(\text{key}) = \text{key} \% \text{size of the hash table}$

$\text{hash_index}(\text{key}) = 52 \% 7 = 3$

What if we have more than one element with the same index? As in our case, 52 and 66 will get the same index, 3. This leads to a collision. There's always a need for a good hash function to avoid collision in frequent cases, but it is likely to happen.

There are two ways to deal with collision:

- Separate chaining
- Open Addressing



Hashing in C++ STL can be done with the help of the hash class, which we are yet to see in the examples given below. When we pass an argument to the hash class, it obtains the hash value of the passed argument. The hashed value makes searching for objects easier. The keys passed as the argument will be mapped to their obtained hash values.

Given below are some programs to find the hash value of different arguments.

Character Hashing

// Program to demonstrate the character hashing.

```
#include <iostream>
```

// To avoid using std:: before each statement.

```
using namespace std;
```

// Function for character hashing.

```
void hashingChar() {
```

```
    char ch = 'n';
```

// Instantiation of a hash object.

```
    hash<char> char_hash;
```

```
// Using operator() to return hashed value.

cout << "The hashed value of character 'n' is: " << char_hash(ch) << endl;

}

// Main function.

int main() {

    // Calling function for character hashing.

    hashingChar();

}
```

Output:

The hashed value is: 110

String Hashing

```
// Program to demonstrate the string hashing.
```

```
#include <iostream>
```

```
// To avoid using std:: before each statement.
```

```
using namespace std;
```

```
// Function for string hashing.
```

```
void hashingString() {
```

```
    string str = "hashing";
```

```
    // Instantiation of a hash object.
```

```
    hash<string> string_hash;
```

```
    // Using operator() to return hashed value.
```

```
    cout << "The hashed value of string is: " << string_hash(str) << endl;
```

```
}
```

```
// Main function.
```

```
int main() {
```

```
    // Calling function for string hashing.
```

```
    hashingString();
```

```
}
```

Output:

The hashed value of the string is: 11997348938128597782

Integer Hashing

```
// Program to demonstrate the integer hashing.
```

```
#include <iostream>
```

```
// To avoid using std:: before each statement.
```

```
using namespace std;
```

```
// Function for integer hashing.
```

```
void hashingInteger() {
```

```
    int num = 17;
```

```
// Instantiation of a hash object.

hash<int> int_hash;


// Using operator() to return hashed value.

cout << "The hashed value of integer is: " << int_hash(num) << endl;

}


// Main function.

int main() {

    // Calling function for string hashing.

    hashingInteger();

}
```

Output:

The hashed value of the integer is: 17

Vector Hashing

```
// Program to demonstrate the vector hashing
```

```
#include <iostream>
```

```
#include <vector>
```

```
// To avoid using std:: before each statement.
```

```
using namespace std;
```

```
// Function for vector hashing.
```

```
void hashingVector() {
```

```
    vector<bool> vec { true, true, false, false };
```

```
    // Instantiation of a hash object.
```

```
    hash<vector<bool>> vector_hash;
```

```
    // Using operator() to return hashed value.
```

```
    cout << "The hashed value of vector_hash is: " << vector_hash(vec) << endl;
```

```
}
```

```
// Main function.

int main() {

    // Calling function for vector hashing.

    hashingVector();

}
```

Output:

The hashed value of vector_hash is: 17166569625921664880

Examples to Understand the Working of Hashing Function in C++

Let us look at a C++ program to see how the hash function works for a hash table. We also need to deal with collision, so each index comprises a list sequence of keys because two or more keys can have the same table position.

And to get the index, we'll use the following hash function, `get_hash()`, which returns the index by division.

```
int get_hash(int key) {

    return key % ht_size;

}
```

Here, `ht_size` is the size of the hash table.

```
// Program to demonstrate the working of the hash function in the table.
```

```
#include <iostream>
```

```
#include <list>
```

```
using namespace std;
```

```
// Creating class hash table.
```

```
class hash_table {
```

```
    // Initializing a list for every index.
```

```
    list<int> *tbl;
```

```
    int ht_size;
```

```
    // Hash function to get the index of the keys.
```

```
    int get_hash(int key) {
```

```
        return key % ht_size;
```

```
    }
```

```
public:
```

```
    // Constructor for the class hash_table.
```

```
hash_table(int n) {

    ht_size = n;

    tbl = new list<int>[ht_size];

}


// Function to insert keys in the hash table.

void insert_key(int key) {

    tbl[get_hash(key)].push_back(key);

}


// Function to display all the keys at their indexes.

void show() {

    for(int i = 0; i < ht_size; i++) {

        cout << "Values at index " << i << ": ";

        for(int num : tbl[i])

            cout << num << " ";

        cout << endl;

    }

}
```

```
};
```

```
// Main function.
```

```
int main() {
```

```
    // Creating an array of keys.
```

```
    int arr[] = {52, 42, 44, 66, 25, 29, 32};
```

```
    // Find the size of the array.
```

```
    int sz_arr = sizeof(arr)/sizeof(int);
```

```
    // Initializing the hash table.
```

```
    hash_table ht(sz_arr);
```

```
    // Inserting the elements into the hash table.
```

```
    for(int i = 0; i < sz_arr; i++) {
```

```
        ht.insert_key(arr[i]);
```

```
    }
```

```
    cout << "The hash table after insertion of elements: " << endl;
```

```
    // Calling function to display the hash table.
```

```
    ht.show();  
  
    return 0;  
}
```

Output:

The hash table after the insertion of elements:

Values at index 0: 42

Values at index 1: 29

Values at index 2: 44

Values at index 3: 52 66

Values at index 4: 25 32

Values at index 5:

Values at index 6:

As evident in the above program, we can have several keys having the same index after division, so both keys have to be associated with the same index. Therefore, a list has to be used at every index to store more keys in the same position.

iostream in C++

iostream stands for standard input-output stream. `#include iostream` declares objects that control reading from and writing to the standard streams. In other words, the iostream library is an object-oriented library that provides input and output functionality using streams.

A stream is a sequence of bytes. You can think of it as an abstraction representing a device. You can perform I/O operations on the device via this abstraction. You must include an iostream header file to input and output from a C++ program.

Syntax

`#include iostream` provides the most used standard input and output streams, `cin` and `cout`. The syntax for using them is as follows:

1. Standard Output Stream -- cout

- It is an instance of the `ostream` class.
- It produces output on the standard output device, i.e., the display screen.
- We need to use the stream insertion operator `<<` to insert data into the standard output stream `cout`, which has to be displayed on the screen.

Syntax:

```
cout << variable_name;
```

OR

```
cout << variable1 << variable2 << ... ;
```

This way of using multiple stream insertion operators with a single `cout` is called cascading. It helps print multiple variables adjacent to each other on the same line.

What if you want to print on separate lines and cascade simultaneously? You can do this in two ways, and both of these can be used with string literals also:

1. Using `\n`, the new line character

Syntax:

```
cout << variable1 << '\n' << variable2
```

It will print variable1's value and print a new line. Finally, it prints the variable2's value on a new line.

2. Using endl, a manipulator

Syntax:

```
cout << variable1 << endl << variable2;
```

A manipulator in C++ changes the behavior of an input or output stream. endl manipulator is a part of the std namespace and can also be used to insert a new line.

2. Standard Input Stream -- cin

- It is an instance of the istream class.
- It reads input from the standard input device, i.e., the keyboard.
- We need to use the stream extraction operator >> to extract data entered using the keyboard.

Syntax:

```
cin >> variable_name;
```

OR

```
cin >> variable1 >> variable2 >> ... ;
```

How Works in C++?

The most basic functionality any C++ program provides is the ability to take input from the user via keyboard and display output on the screen. You must understand input and output streams to understand how this data flow works through the objects defined in `#include iostream`.

1. Input Stream

- It involves the flow of bytes from the device to the main memory. This process is known as input.
- An input stream object (like `cin`) is a source of bytes.
- The most important input stream classes are `istream`, `ifstream`, and `istringstream`.

2. Output Stream

- It involves the flow of bytes from the main memory to the device. This process is known as output. It is opposite to how the input stream works.
- An output stream object (like `cout`) is a destination for bytes.
- The most important output stream classes are `ostream`, `ofstream`, and `ostringstream`.

istream

The C++ `istream` class provides a set of functions and operators for reading data from various input sources.

Before using the `istream` class, we need to include the `<iostream>` header in our program.

```
#include <iostream>
```

Example 1: Reading Data From the Console

To read data from the console using istream, you can use the cin object with the extraction operator >>. Here's an example:

```
#include <iostream>

using namespace std;

int main() {

    int entered_number;

    cout << "Enter an integer: ";

    // read data from the console
    cin >> entered_number;

    cout << "You entered: " << entered_number << endl;

    return 0;
}
```

Output

```
Enter an integer: 3
```

```
You entered: 3
```

The get() Function

The `get()` function of the `istream` class is primarily used for reading individual characters from the input stream.

Key Characteristics

- **Reading Characters:** It reads the next character from the input stream and advances the stream's internal position indicator.
- **Single Character:** Unlike the `>>` operator, `get()` reads characters as they are, including spaces, tabs, and newline characters.
- **Character Extraction:** You can use it to extract characters into variables of type `char` or to store them in character arrays (C-strings).
- **Delimiter Control:** It allows you to specify a delimiter character as an optional parameter. When you specify a delimiter, `get()` stops reading characters when it encounters that delimiter. This is useful for reading words or lines.

Example 2: C++ get() Function

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    char ch;
```

```
    cout << "Enter a sentence: ";
```

```
    // read the character one by one
```

```
    // until a new line is encountered
```

```
while (cin.get(ch) && ch != '\n') {  
    cout << ch;  
}  
  
return 0;  
}  
Output
```

```
Enter a sentence: Hello World  
Hello World
```

C++ **getline()** Function

The `getline()` function in `istream` is used to read a line of text from the specified input stream (such as `cin` for standard input).

Key Characteristics

- **Reading Lines:** Unlike the `get()` function, `getline()` reads entire lines of text, up to a specified delimiter or the end of the line.
- **Delimiter Control:** You can specify a delimiter character like one that indicates the end of a line. This allows you to read multiple lines of text sequentially.
- **Buffer Size:** It accepts an optional parameter to specify the maximum number of characters to read, preventing buffer overflows and handling long lines gracefully.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    string line;
```

```
    cout << "Enter a line of text: ";
```

```
    // read a line of input from user
```

```
    getline(cin, line);
```

```
    cout << "You entered: " << line << endl;
```

```
    return 0;}
```

Output

```
Enter a line of text: Hello
```

```
You entered: Hello
```

Assignment : Read and write about iomanip, ifstream, ostream, ofstream, etc.

